Supervised learning with missing values and imputation

Marine Le Morvan – Soda, INRIA

Supervised learning and missing values

Regression and classification tasks are solved by minimizing a risk:

$$f^\star = \underset{f}{\operatorname{argmin}} \quad \mathbb{E}_{X,Y}\left[\ell\left(f(X), Y\right)\right]$$

with inputs X and labels Y. We care about good **predictions**.

$$\begin{pmatrix} 1.2 & \texttt{na} & 0.2 & -0.4 \\ -0.3 & \texttt{na} & \texttt{na} & -0.9 \\ \texttt{na} & 0.4 & \texttt{na} & \texttt{na} \\ -2.8 & 1.9 & 1.6 & 2.2 \\ \texttt{na} & 1.7 & -2.4 & \texttt{na} \end{pmatrix} \quad \begin{pmatrix} 1.7 \\ -0.2 \\ 1.6 \\ -2.2 \\ 0.8 \end{pmatrix}$$

Incomplete X Complete Y

Questions:

- How to deal with missing values in the **train** set?
- How to deal with missing values in the **test** set? (prediction time)

Challenges of supervised learning with missing values

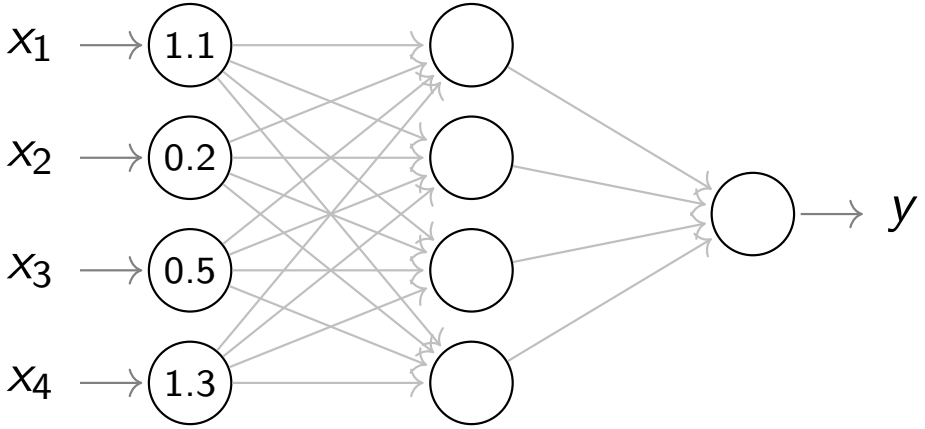
- Supervised learning with missing values:
  - Scarce literature (vs Inference, Imputation)

Challenges of supervised learning with missing values

- Supervised learning with missing values:
  - Scarce literature (vs Inference, Imputation)
  - Arbitrary subsets of variables.

Samples

$$\begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ & & & \\ & & & \end{bmatrix}$$

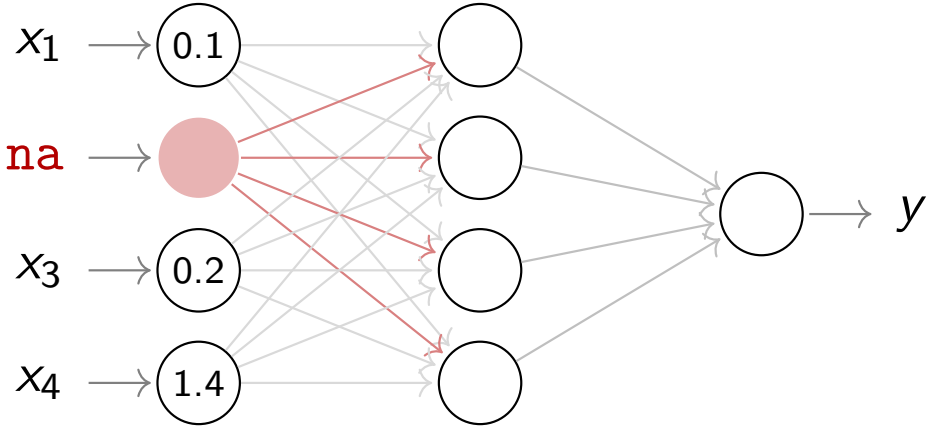


Challenges of supervised learning with missing values

- Supervised learning with missing values:
  - Scarce literature (vs Inference, Imputation)
  - Arbitrary subsets of variables.

Samples

$$\begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ 0.1 & \texttt{na} & 0.2 & 1.4 \\ & & & \end{bmatrix}$$

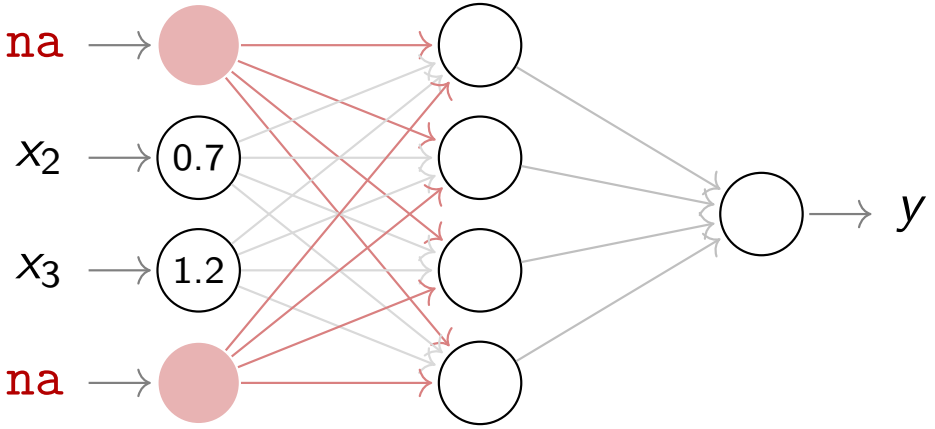


Challenges of supervised learning with missing values

- Supervised learning with missing values:
  - Scarce literature (vs Inference, Imputation)
  - Arbitrary subsets of variables.

Samples

$$\begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ 0.1 & \texttt{na} & 0.2 & 1.4 \\ \texttt{na} & 0.7 & 1.2 & \texttt{na} \end{bmatrix}$$

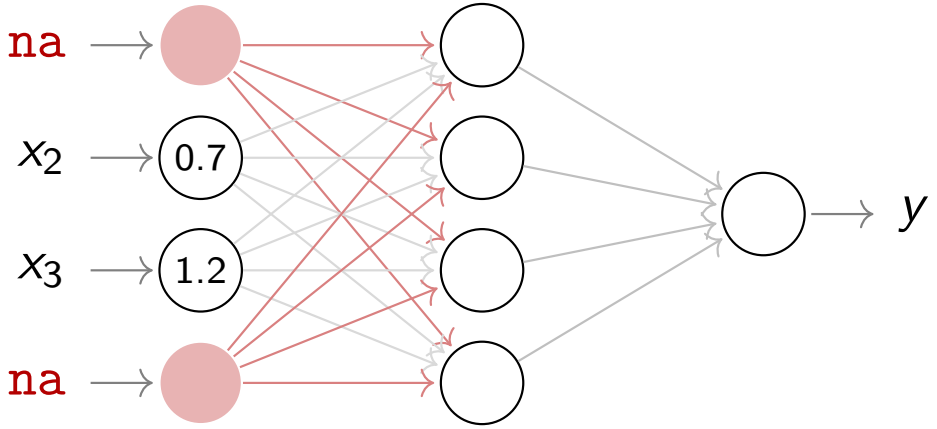


Challenges of supervised learning with missing values

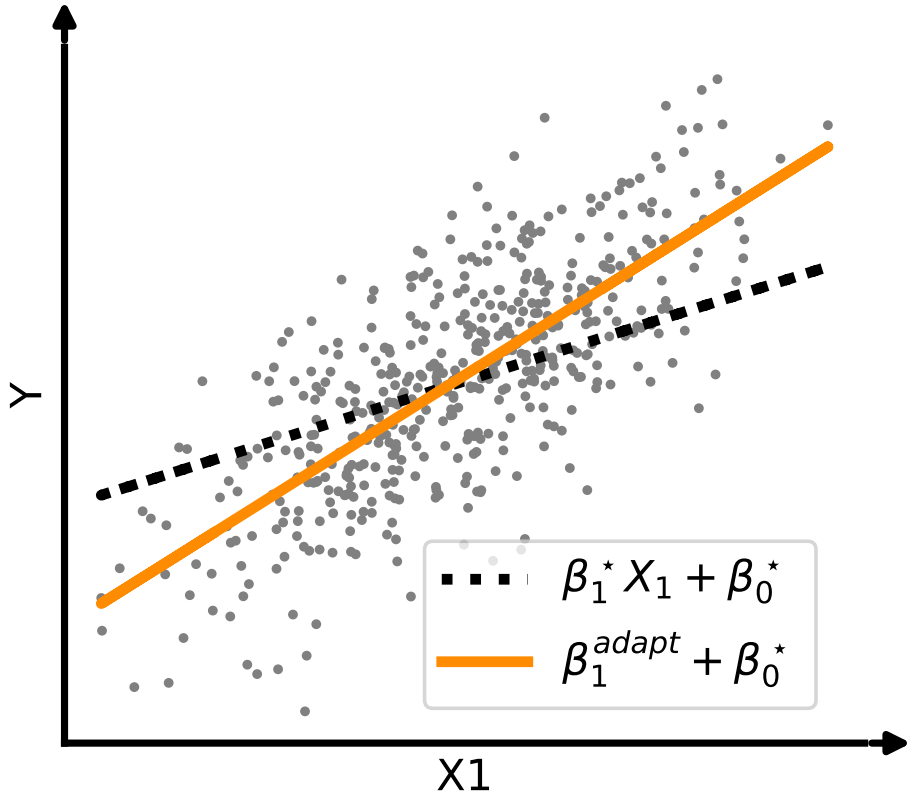
- Supervised learning with missing values:
  - Scarce literature (vs Inference, Imputation)
  - Arbitrary subsets of variables.
  - Computational and statistical challenge.
    Ex: $d = 50 \implies 2^{50} \approx 10^{15}$ possible missing data patterns.

Samples

$$\begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ 0.1 & \texttt{na} & 0.2 & 1.4 \\ \texttt{na} & 0.7 & 1.2 & \texttt{na} \end{bmatrix}$$



The case of linear regression with missing values: intuition



$$Y = \beta_1^\star X_1 + \beta_2^\star X_2$$

$$\text{cor}(X_1, X_2) = 0.5.$$

If X_2 is missing, the coefficient of X_1 should **compensate for the missingness of X_2**.

For a general problem in d dimensions, there are 2^d **possible missing data patterns**, and thus 2^d slopes to estimate.

How do people usually handle missing values?

Widespread current practice: Impute-then-Regress (or classify)

$$\begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ 0.1 & \texttt{na} & 0.2 & 1.4 \\ \texttt{na} & 0.7 & 1.2 & \texttt{na} \end{bmatrix} \xrightarrow{\textbf{Impute}} \begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ 0.1 & 0.5 & 0.2 & 1.4 \\ 0.6 & 0.7 & 1.2 & 1.3 \end{bmatrix} \xrightarrow{\textbf{Regress}} \text{Predictions } \mathbf{y}$$

Important note: The imputation should be learned on the **train set only**, before being applied to the train and test sets.

How do people usually handle missing values?

Widespread current practice: Impute-then-Regress (or classify)

$$\begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ 0.1 & \texttt{na} & 0.2 & 1.4 \\ \texttt{na} & 0.7 & 1.2 & \texttt{na} \end{bmatrix} \xrightarrow{\textbf{Impute}} \begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ 0.1 & 0.5 & 0.2 & 1.4 \\ 0.6 & 0.7 & 1.2 & 1.3 \end{bmatrix} \xrightarrow{\textbf{Regress}} \text{Predictions } \mathbf{y}$$

Important note: The imputation should be learned on the **train set only**, before being applied to the train and test sets.

Let's go through a guided tour of imputation strategies.

A review of imputation strategies

Imputation

- i.e. filling in with likely values.

$$\begin{pmatrix} 1.2 & \texttt{na} & 0.2 & -0.4 \\ -0.3 & \texttt{na} & \texttt{na} & -0.9 \\ \texttt{na} & 0.4 & \texttt{na} & \texttt{na} \\ -2.8 & 1.9 & 1.6 & 2.2 \\ \texttt{na} & 1.7 & -2.4 & \texttt{na} \end{pmatrix}$$

- Naive imputations (e.g. constants).
- k-Nearest Neighbors.
- Likelihood-based approaches (EM).
- Multiple imputation by chained equation (MICE).
- Matrix factorization.
- Many recent approaches based on GANs, VAEs, GNNS, optimal transport, ...

Some simple imputation techniques

- **Univariate imputations**:
  - by a constant (e.g. 0).
  - by the mean or the median (for continuous features).
  - by the mode (for categorical variables).
- **k-Nearest Neighbors imputation**:
  - Nearest neighbours found based on observed features only.
  - Imputation using the mean value of the nearest neighbours.
- **Implementation in Python**:
  - Scikit-learn's `SimpleImputer` and `KNNImputer`.

Likelihood-based approach - 1/2

- We need to assume a **parametric model** for the probability density of X, with parameters θ:
$$X \sim p_\theta(X).$$
For example, we can assume the data follows a Gaussian distribution with parameters $\theta = (\mu, \Sigma)$.

- With complete data, the maximum likelihood estimator (MLE) $\hat{\theta}$ is given by:
$$\hat{\theta} = \underset{\theta}{\text{argmax}} \prod_{i=1}^{n} p_\theta(X_i)$$

- In the presence of missing data, under **MAR** hypothesis, we showed that:
  - the missingness mechanism can be **ignored** and that consequently:
$$\hat{\theta} = \underset{\theta}{\text{argmax}} \prod_{i=1}^{n} p_\theta(X_{i,obs})$$
  - $\hat{\theta}$ can be obtained using an **EM algorithm**.

Likelihood-based approach - 2/2

- Once $\hat{\theta}$ is obtained, it is possible to **impute using** $p_{\hat{\theta}}(X_{mis}|X_{obs})$.
  - deterministically using the conditional expectation $\mathbb{E}\left[X_{mis}|X_{obs}\right]$.
  - or stochastically with random draws from $p_{\hat{\theta}}(X_{mis}|X_{obs})$.

- For p_θ, the **multivariate Gaussian distribution** is most widely applied. It has the advantage of being analytically tractable.

  Indeed, if $p_\theta(X) = \mathcal{N}\left(X; \mu, \Sigma\right)$, the conditional Gaussian formula tells us that $p_\theta(X_{mis}|X_{obs})$ is also Gaussian with mean and covariance:

$$\mu_{mis|obs} = \mu_{mis} + \Sigma_{mis,obs}\Sigma_{obs}^{-1}\left(X_{obs} - \mu_{obs}\right) \tag{1}$$

$$\Sigma_{mis|obs} = \Sigma_{mis,mis} - \Sigma_{mis,obs}\Sigma_{obs}^{-1}\Sigma_{obs,mis} \tag{2}$$

- Implementation:
  - Python: no widely accepted package.
  - R: package `norm` (assumes a Gaussian distribution).

MICE - 1/2

- Standard multivariate distributions (e.g. Gaussian) do not always provide a good fit for real data.
- **Idea**: Specify a **conditional distribution for each variable** X_j depending on all others.
- MICE is an **iterative algorithm**:
  1. For each $j \in \left[1, \ldots, J\right]$, specify :

$$p_{\theta_j}(X_j | X_1, \ldots, X_{j-1}, X_{j+1}, \ldots, X_J)$$

  2. Create initial approximate imputations using a simple procedure.
  3. For a number of iterations T:
     For each variable j in turn:
     ✓Update θ_j .
     ✓Update imputations with random draws from $p_{\theta_j}(X_j | \ldots)$.
- We can also define a deterministic version of MICE.

MICE - 1/2

- Standard multivariate distributions (e.g. Gaussian) do not always provide a good fit for real data.
- **Idea**: Specify a **conditional distribution for each variable** X_j depending on all others.
- MICE is an **iterative algorithm**:
  1. For each $j \in \left[1, \ldots, J\right]$, specify a regression model:

     $$X_j = f_{\theta_j}(X_1, \ldots, X_{j-1}, X_{j+1}, \ldots, X_J).$$

  2. Create initial approximate imputations using a simple procedure.
  3. For a number of iterations T:

     For each variable j in turn:

     ✓ Learn θ_j.

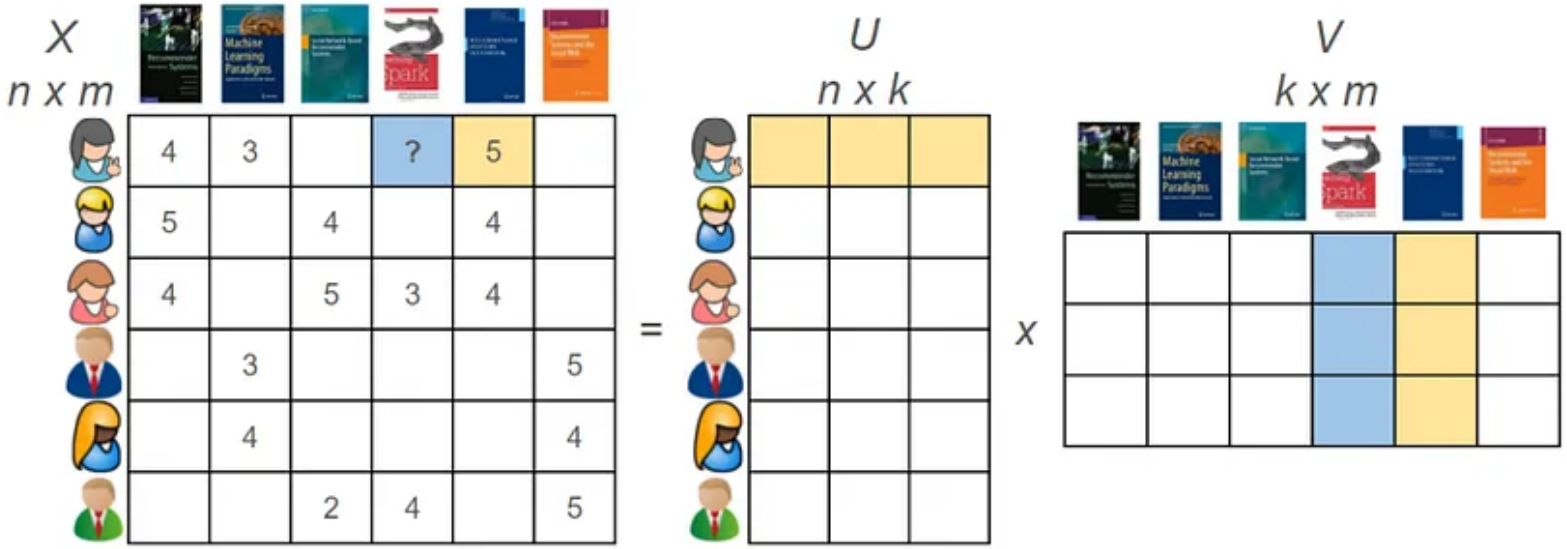
     ✓Update imputations with predictions $f_{\theta_j}(X_1, \ldots, X_{j-1}, X_{j+1}, \ldots, X_J)$.
- We can also define a deterministic version of MICE.

MICE - 2/2

- Advantages:
  - Can handle any type of variable (continuous, binary, categorical). Just define the conditional distribution/regression function accordingly.
  - State-of-the-art performances, notably with `missforest`, a deterministic version where random forest are used for regressions.
- Inconveniences:
  - Questions about the convergence.
  - Computationally expensive.
- Implementation:
  - In Python: Scikit-learn's `IterativeImputer`.
  - in R: `mice`, `missforest` (+ other options).

Imputation by Matrix Factorization

- **Idea**: Find latent factors to reconstruct well the observed values of a matrix.



- Many techniques coming from the literature on Recommender systems.

- Many algorithms:
  - PCA with missing values (EM algorithm again)
  - Iterated SVD
  - Alternated Least squares
  - Funk SVD (stochastic gradient descent on the reconstruction error)
  - ...

A jungle of recent imputation methods

Missing Data Imputation using Optimal Transport

MIWAE: Deep Generative Modelling and Imputation of Incomplete Data Sets

Handling Missing Data with
Graph Representation Learning

Missing Value Imputation for Mixed Data via Gaussian Copula

GAIN: Missing Data Imputation using Generative Adversarial Nets

and many others!

How do people usually handle missing values?

Widespread current practice: Impute-then-Regress (or classify)

$$\begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ 0.1 & \texttt{na} & 0.2 & 1.4 \\ \texttt{na} & 0.7 & 1.2 & \texttt{na} \end{bmatrix} \xrightarrow{\textbf{Impute}} \begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ 0.1 & 0.5 & 0.2 & 1.4 \\ 0.6 & 0.7 & 1.2 & 1.3 \end{bmatrix} \xrightarrow{\textbf{Regress}} \text{Predictions } \mathbf{y}$$

Important note: The imputation should be learned on the **train set only**, before being applied to the train and test sets.

How do people usually handle missing values?

Widespread current practice: Impute-then-Regress (or classify)

$$\begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ 0.1 & \texttt{na} & 0.2 & 1.4 \\ \texttt{na} & 0.7 & 1.2 & \texttt{na} \end{bmatrix} \xrightarrow{\textbf{Impute}} \begin{bmatrix} 1.1 & 0.2 & 0.5 & 1.3 \\ 0.1 & 0.5 & 0.2 & 1.4 \\ 0.6 & 0.7 & 1.2 & 1.3 \end{bmatrix} \xrightarrow{\textbf{Regress}} \text{Predictions } \mathbf{y}$$

Important note: The imputation should be learned on the **train set only**, before being applied to the train and test sets.

Open research question:

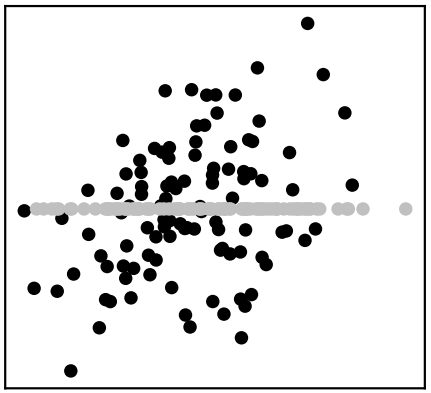
Do we need to impute well to predict well?

Do we need to impute well to predict well?

- **Theoretically -and asymptotically-, no**. A sufficiently flexible learner can be Bayes optimal (Le Morvan et al., 2021).

Inference ✗

Distorted distribution



mean imputation

Supervised learning
✓

Bayes optimality

This is true *whatever the missing data mechanism*.

- **In practice, it depends.** Illustration with a benchmarking study on health databases Perez-Lebel et al. (2022).

13 tasks across 4 health databases

traumabase.eu

- $n \approx 10,000$
- 5 prediction tasks:
  - death (C)
  - hemorrhagic shock x2 (C)
  - platelet level (R)
  - septic shock (C)

biobank uk
Improving the health of future generations

- $n \approx 300,000$
- 5 prediction tasks:
  - malignant breast neoplasm x2 (C)
  - fluid intelligence score (R)
  - Parkinson (C)
  - Melanoma (C)

MIMIC

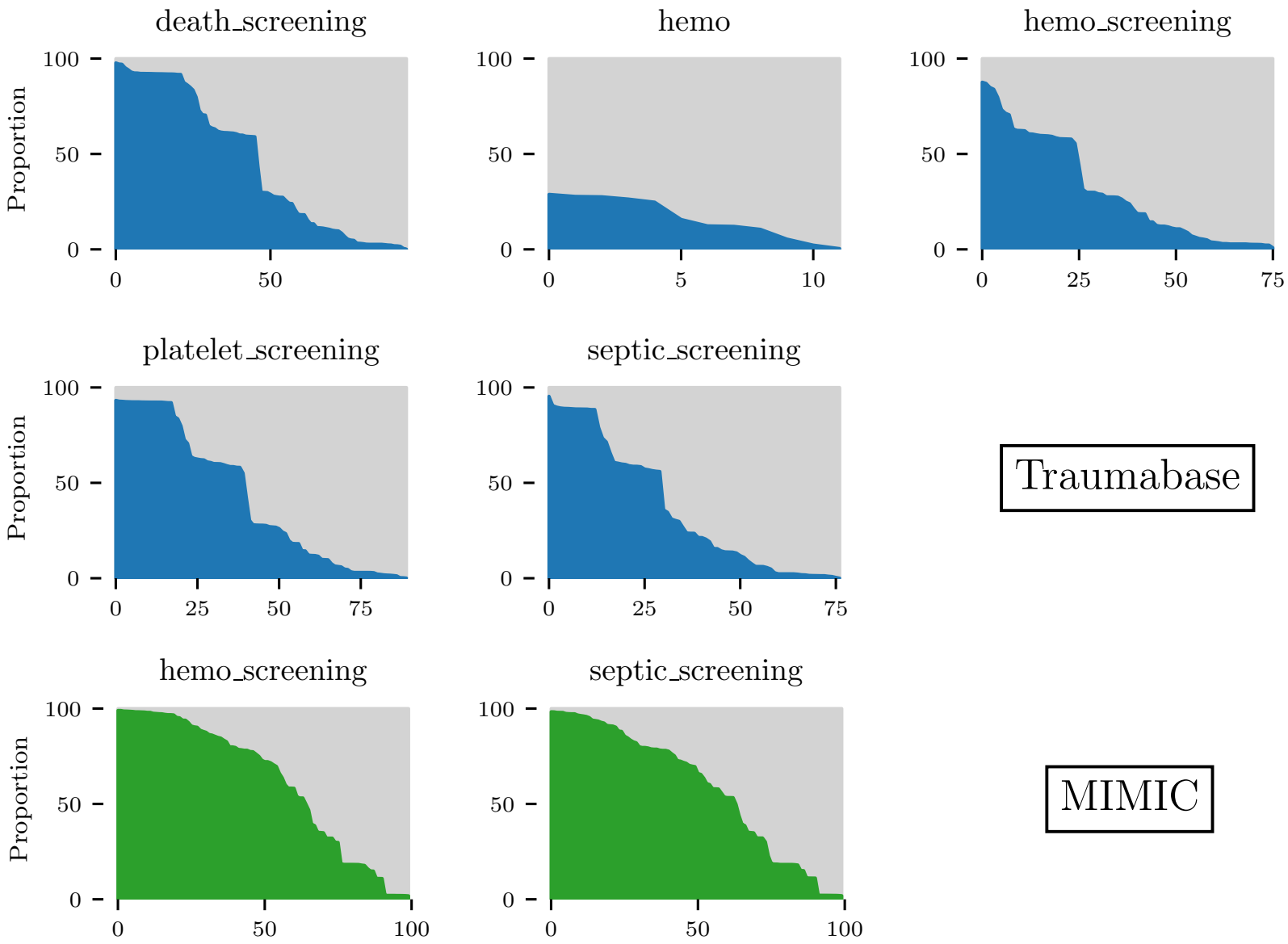
- $n \approx 30,000$
- 2 prediction tasks:
  - hemorrhagic shock (C)
  - septic shock (C)

NHIS
NATIONAL HEALTH INSURANCE SCHEME

- $n \approx 20,000$
- 1 prediction task:
  - yearly income (R)

For all databases, the number of features considered is between 12 and 100.

Missingness levels on MIMIC and the Traumabase



Proportion of missing values for each feature.

Baselines benchmarked

Predictive models

- Gradient Boosted Regression Trees
- Linear models (not shown)

Missing values handling

- Constant imputation
  - mean
  - median
- Conditional imputation
  - Iterative Imputer (MICE like)
  - K-Neareast Neighbours (kNN)
- MIA: Missing Incorporated in Attribute
  - only works with trees
  - does an implicit imputation

Baselines benchmarked

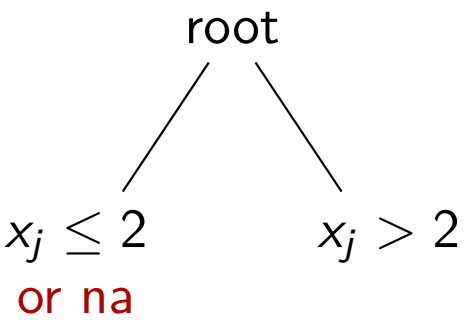
Predictive models

- Gradient Boosted Regression Trees
- Linear models (not shown)

Missing values handling

- Constant imputation
  - mean
  - median
- Conditional imputation
  - Iterative Imputer (MICE like)
  - K-Neareast Neighbours (kNN)
- MIA: Missing Incorporated in Attribute
  - only works with trees
  - does an implicit imputation

MIA:



Option 1

Baselines benchmarked

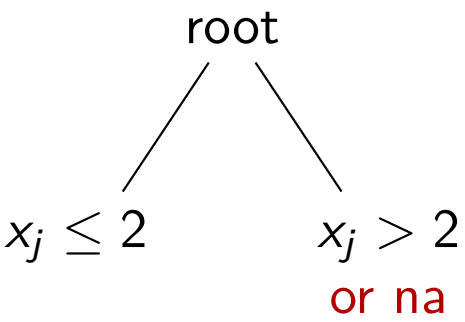
Predictive models

- Gradient Boosted Regression Trees
- Linear models (not shown)

Missing values handling

- Constant imputation
  - mean
  - median
- Conditional imputation
  - Iterative Imputer (MICE like)
  - K-Neareast Neighbours (kNN)
- MIA: Missing Incorporated in Attribute
  - only works with trees
  - does an implicit imputation

MIA:



Option 2

Baselines benchmarked

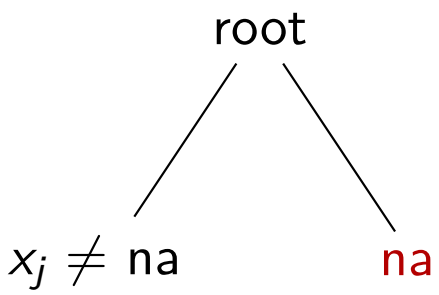
Predictive models

- Gradient Boosted Regression Trees
- Linear models (not shown)

Missing values handling

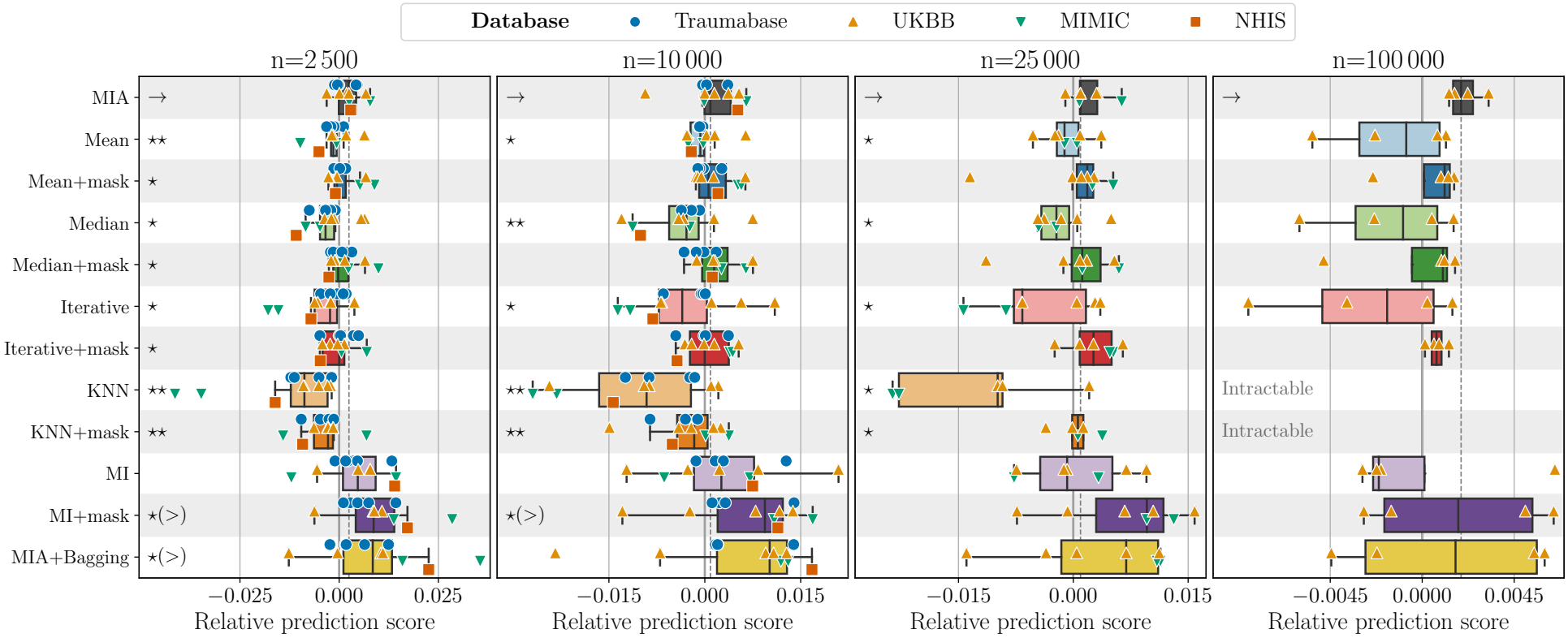
- Constant imputation
  - mean
  - median
- Conditional imputation
  - Iterative Imputer (MICE like)
  - K-Neareast Neighbours (kNN)
- MIA: Missing Incorporated in Attribute
  - only works with trees
  - does an implicit imputation

MIA:

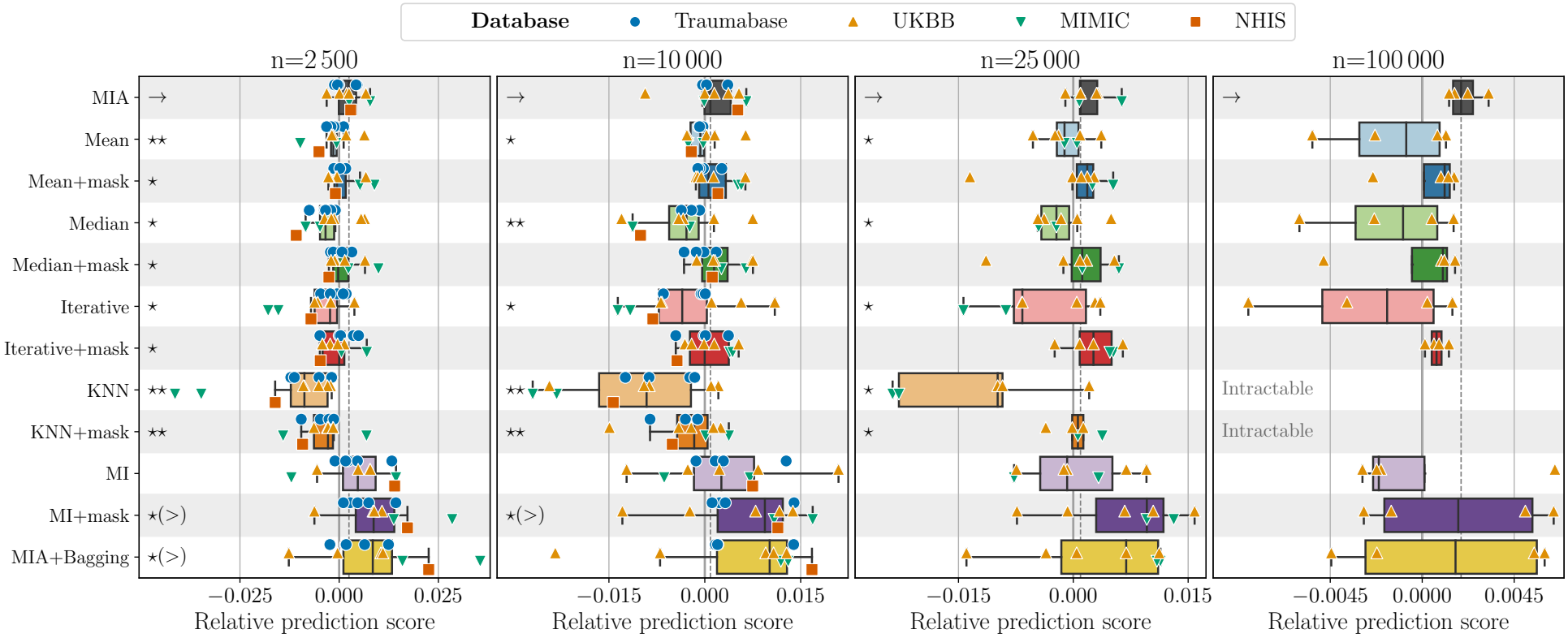


Option 3

Prediction performances

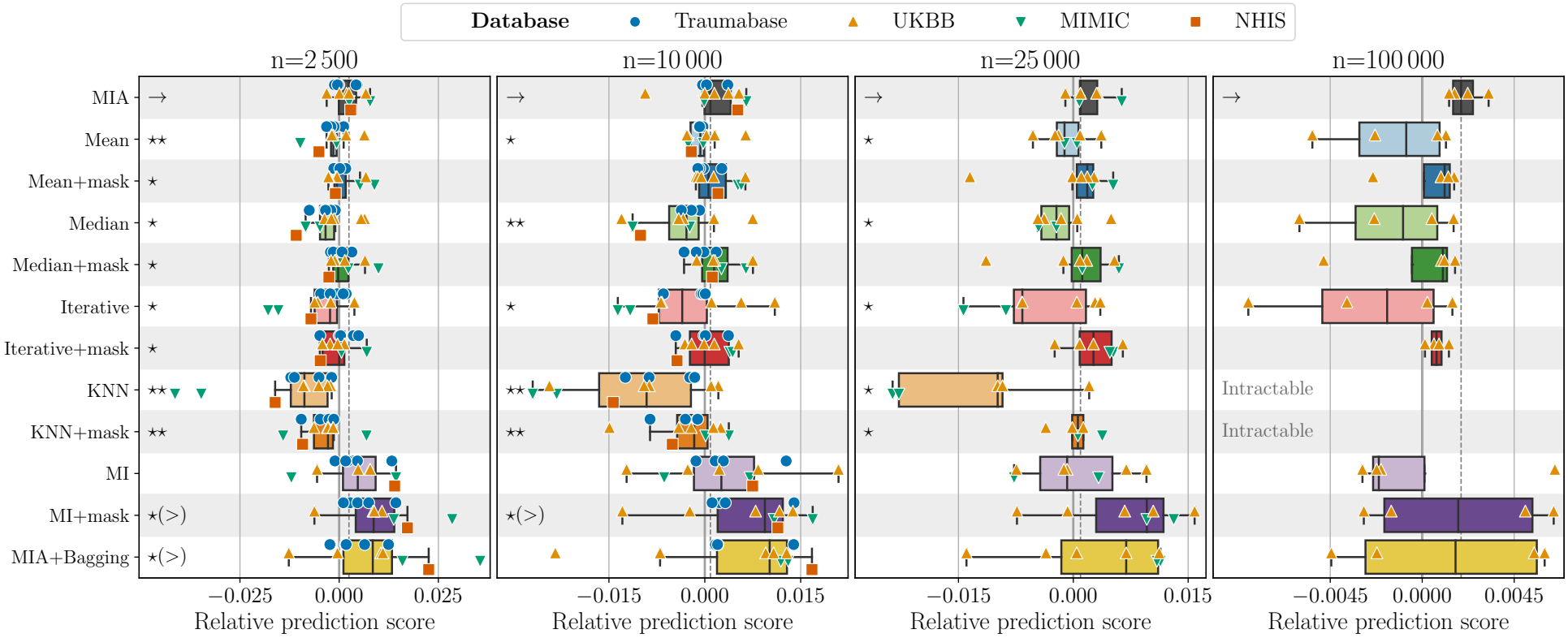


Prediction performances



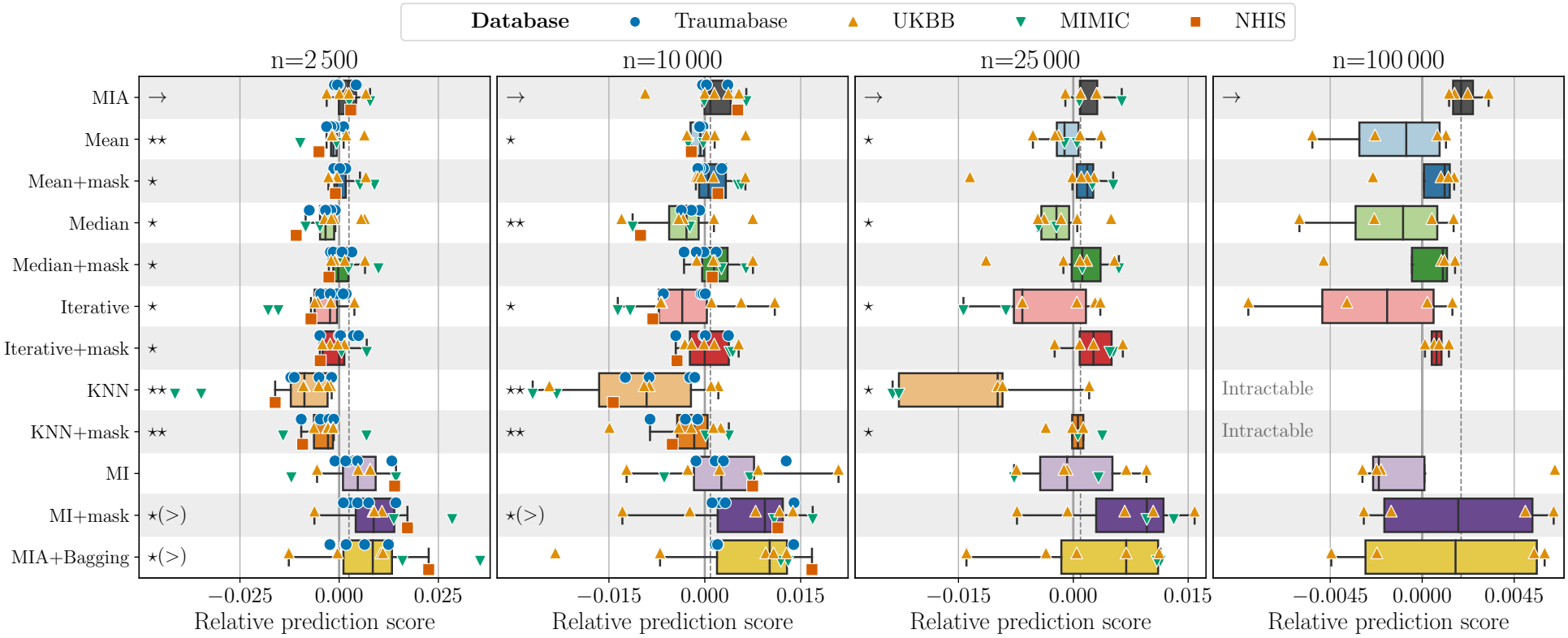
- Adding the mask improves prediction ($\Longrightarrow$ informative missingness).

Prediction performances



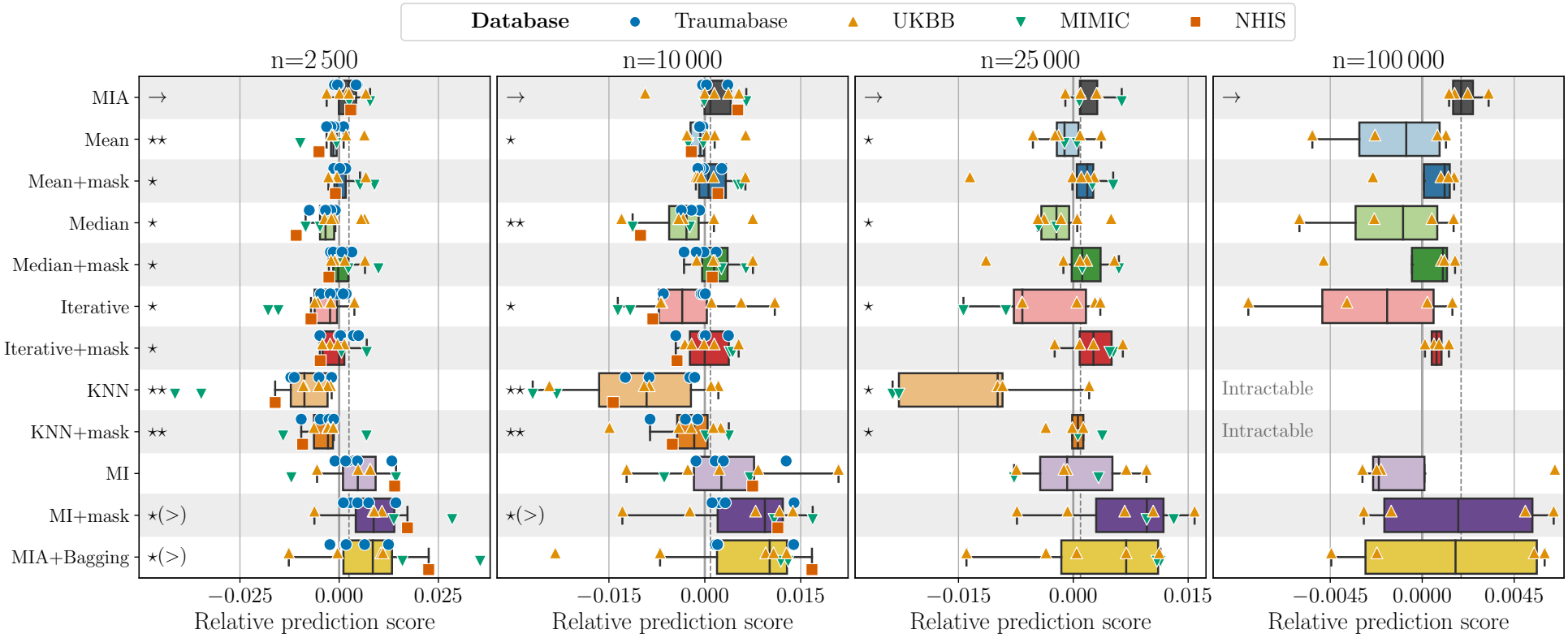
- Adding the mask improves prediction ($\Longrightarrow$ informative missingness).
- Conditional imputation is on par with constant imputation

Prediction performances



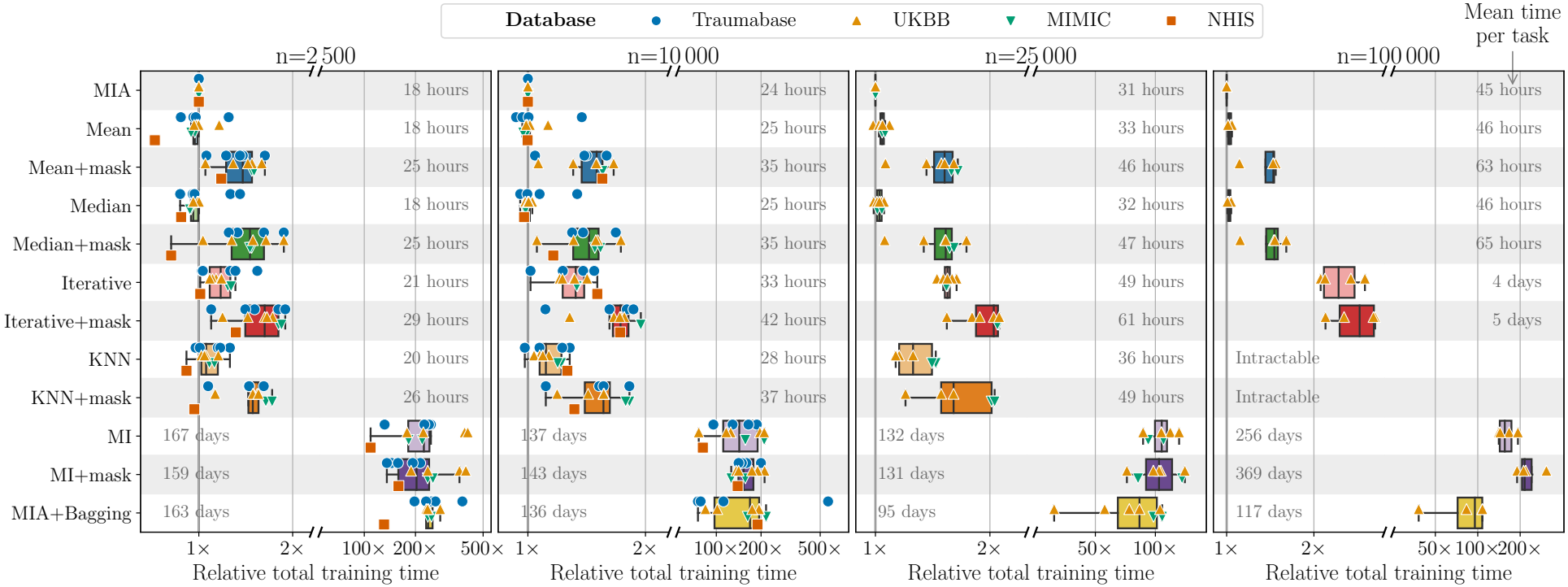
- Adding the mask improves prediction ($\Longrightarrow$ informative missingness).
- Conditional imputation is on par with constant imputation
- Native missing-values support with MIA gives better predictions than imputation.

Prediction performances



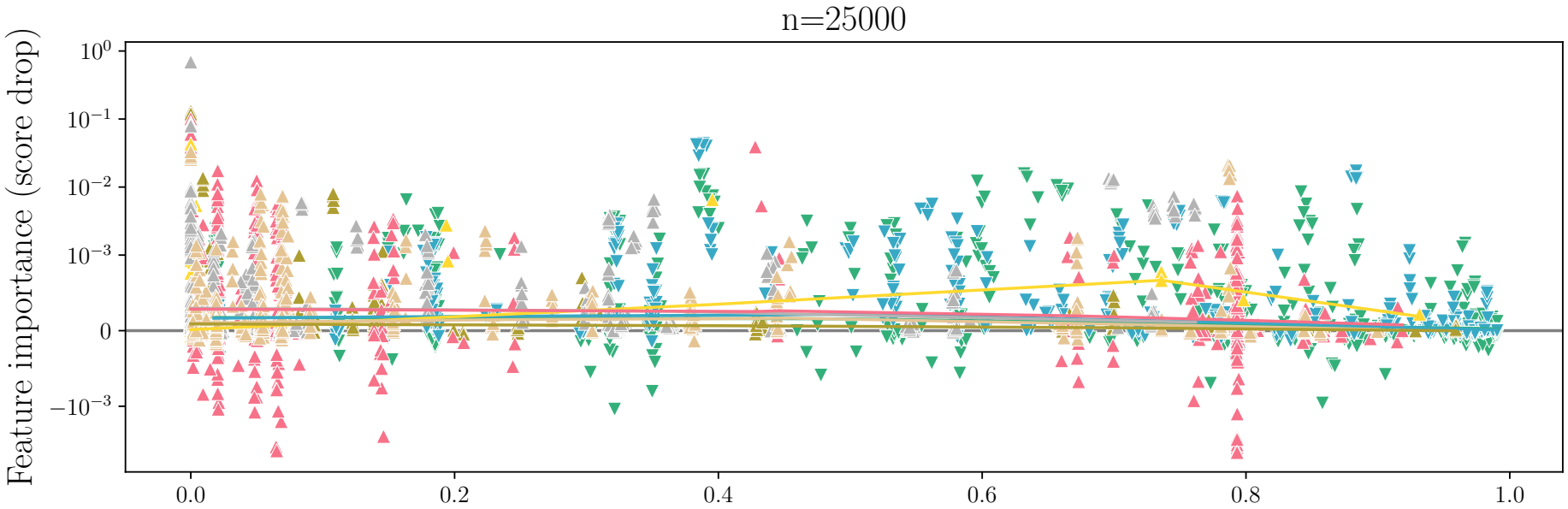
- Adding the mask improves prediction ($\Longrightarrow$ informative missingness).
- Conditional imputation is on par with constant imputation
- Native missing-values support with MIA gives better predictions than imputation.
- Bagging improves prediction performance but with a prohibitive time cost.

Prediction performances



- Adding the mask improves prediction ($\Longrightarrow$ informative missingness).
- Conditional imputation is on par with constant imputation
- Native missing-values support with MIA gives better predictions than imputation.
- Bagging improves prediction performance but with a prohibitive time cost.

Features with high missing rates are also important



Feature importances as a function of missingness proportion.

Summary and takeaways

- Impute-then-Regress (or classify) widely used in practice.
- The imputation algorithm should be learned on the train set only.
- MICE (and in particular missforest) are state-of-the-art imputers.
- Better imputations sometimes, but not always, improve predictions.
- Use the missingness indicator as additional input features (option `add_indicator` in scikit-learn's imputers).
- Tree-based models with MIA often provide very good performances.
- Ongoing works to handle missing values without an imputation step in neural networks.

Thanks for you attention.

Any questions?

Le Morvan, M., Josse, J., Scornet, E., and Varoquaux, G. (2021). What'sa good imputation to predict with missing values? *Advances in Neural Information Processing Systems*, 34:11530–11540.

Perez-Lebel, A., Varoquaux, G., Le Morvan, M., Josse, J., and Poline, J.-B. (2022). Benchmarking missing-values approaches for predictive models on health databases. *GigaScience*, 11.